



Отчёт о запуске системы тестирования

Исследование продукта, автоматизация, конфигурации и дефекты

Рабочий отчёт

Срез на 14 августа 2026

Semaphore UI v2.19.7 | Bookwright v1.4.0

1. Резюме

Что уже создано и какой практический результат получен

2 192 issues обработано	7 профилей окружения	9 + 1 core + runner сценариев	64 framework self-tests
-----------------------------------	--------------------------------	---	-----------------------------------

Главный результат

Создан не разовый smoke, а поддерживаемая основа тестирования: единый Java-проект, воспроизводимые Docker-профили, typed fixtures, безопасная работа с секретами, автоматический cleanup и диагностика через Allure.

К настоящему моменту:

- проанализированы все 865 открытых и 1 327 закрытых GitHub issues; сформирован реестр и карта компонентов;
- поднят локальный Semaphore UI v2.19.7 и создан единый lifecycle запуска, тестирования и очистки профилей;
- автоматизирован критический поток от login и проекта до Git, task output, schedules, RBAC и stop/force-stop;
- одна core suite подтверждена на SQLite, PostgreSQL, MySQL 8.4 и MariaDB 10.11;
- production-like поток подтверждён через отдельный persistent remote runner;
- реализована двухфазная проверка обновления N-1 -> current и найден блокирующий дефект v2.19.6 -> v2.19.7;
- выявлены расхождения OpenAPI с фактическим router и описаны следующие приоритеты развития.

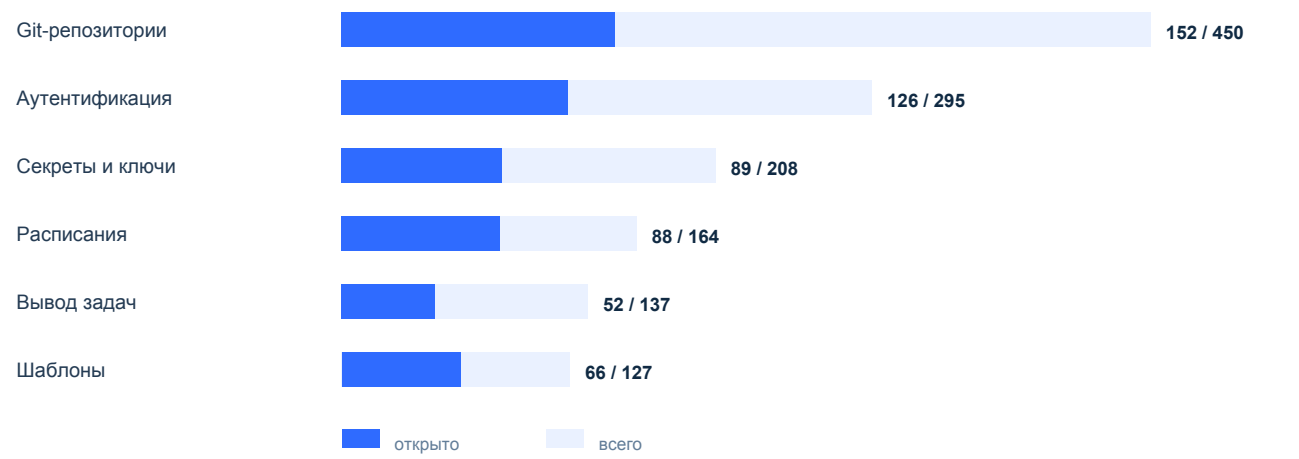
Текущий статус

Базовая функциональная и конфигурационная матрица зелёная. Upgrade-профили намеренно красные до исправления подтверждённого дефекта продукта.

2. Исследование продукта и приоритеты

Решения по автоматизации основаны на фактических обращениях пользователей

Первичный проход выполнен по всем **2 192 issues**. Для каждой записи сохранены дата, ссылка, пользовательская жалоба, наблюдаемый симптом, статус исправления, способ исправления при наличии доказательств и основной компонент системы.



Что показал анализ

Компонент	Всего	Открыто	Основные риски
Git-репозитории	450	152	Clone/pull, SSH/HTTPS, ветки и refs, кеш и пути playbook
Аутентификация	295	126	Login, tokens, LDAP, OIDC, TOTP, sessions и RBAC
Секреты и ключи	208	89	Access keys, Variable Groups, Vault и передача credentials
Расписания	164	88	Cron, timezone/DST, run once и параметры запуска
Вывод задач	137	52	Порядок output, события, диагностика и утечки
Шаблоны	127	66	Связи с repository, inventory, secrets и survey variables

Приоритет P0/P1

Login и project isolation; Git и access keys; inventory и task template; lifecycle и output; schedules; роли manager/task_runner/guest; отсутствие секретов в API и артефактах.

3. Стратегия и карта компонентов

Минимальная система, которую реально поддерживать одному инженеру

Уровень	Роль	Назначение	Статус
API	Основной уровень	Бизнес-логика, RBAC, CRUD, async lifecycle, ошибки	ГОТОВО
Integration	Docker profiles	DB dialect, migrations, runner boundary, Git fixture	ГОТОВО
UI e2e	Выборочно	Только критичные пользовательские пути	ДАЛЕЕ
Release	N-1 -> current	Persistence, schema compatibility, повторный task run	БАГ НАЙДЕН
CI	PR/nightly/release	Разделение по стоимости и стабильности	ПРЕДСТОИТ

Компонентная модель

Группа	Что входит	Ключевой риск
AUTH / USERS	Login, sessions, users, LDAP/OIDC, MFA	Аутентификация, account mapping, session boundaries
PROJECTS / RBAC	Projects, members, roles, permissions	Privilege escalation, cross-project access
TASKS / RUNNERS	Queue, execution, stop, output, runners	Stuck jobs, wrong executor, lost output
REPOSITORIES	Git URL, refs, SSH/HTTPS, cache	Clone failures, credentials, wrong branch
SECRETS	Access keys, Vault, Variable Groups	Leakage, encryption, broken references
SCHEDULES	Cron, run once, timezone, parameters	Missed/duplicate runs, lost variables
DB / MIGRATIONS	SQLite, MySQL, MariaDB, PostgreSQL	Schema drift, upgrade and compatibility
CONFIG / DEPLOY	Compose, packages, Helm, proxy, HA	Environment-specific failures

Принцип

Мы не умножаем каждый тест на каждую конфигурацию. Одна core suite запускается на нескольких опорных профилях, а OIDC, LDAP, HA и специальные executors получают собственные feature-наборы.

4. Техническая основа автоматизации

Bookwright v1.4.0 адаптирован к доменной модели Semaphore

Слой	Реализация	Практическая польза
Tests	JUnit 5 + AssertJ + Allure	Читаемые сценарии, assertions и диагностируемые отчёты
Steps	Разделение target/domain	Project, users, keys, repositories, inventory, templates, tasks, schedules, runners
API clients	Retrofit + OkHttp	Typed contracts, timeout policy, безопасные HTTP attachments
Fixtures	Typed immutable data	Данные не скрытаны в steps; deterministic replay через seed
State	Typed TestStore + preconditions	Явное владение состоянием и изоляция параллельных тестов
Cleanup	Method-scoped LIFO	Удаление ресурсов в обратном порядке без потери исходной ошибки
Environment	Docker Compose profiles	Одинаковый lifecycle локально и в будущей CI

Сквозной ресурсный поток



Защита тестового контура

- Ansible выполняется только из доверенного локального Git fixture, смонтированного read-only.
- Уникальный password marker проверяется на отсутствие в create/get/list API, structured/raw output, Allure и JUnit artifacts.
- Ошибки Git clone сохраняют полезную диагностику, но login/password не попадают в output.
- 64 framework self-tests контролируют waits, state isolation, cleanup, HTTP redaction, fixtures и архитектурные границы.

5. Реализованное продуктивное покрытие

Не количество endpoint, а критические пользовательские и технические контракты

Область	Проверяемый контракт	Результат
Core workflow	Health, invalid/valid login, project, key, repository, inventory, template, task, output, schedule, cleanup	PASS
Secrets	Encrypted login_password key используется в задаче; plaintext отсутствует во всех контролируемых каналах	PASS
Git	Явная ветка, отсутствующий ref, недоступный authenticated HTTPS remote	PASS
RBAC	manager и task_runner: разрешённый запуск, запреты resource/project/member mutations	PASS
Isolation	guest читает назначенный проект, не видит неназначенный и не создаёт access key	PASS
Task lifecycle	Обычный stop и force-stop после фактического старта; terminal stopped; следующий шаг не выполняется	PASS
Remote runner	registered, active, default, online, heartbeat и выполнение всей task suite вне server process	PASS
Release upgrade	Seed на N-1, сохранение DB, current image, поиск данных, старый output, повторный запуск	PRODUCT FAIL

Что подтвердили негативные сценарии

Сценарий	Зафиксированное поведение
Неверный пароль	401, сессия не создаётся
Guest mutation	403 на создание access key
Чужой проект	404 для неназначенного пользователя
Missing Git ref	Task заканчивается error с понятной clone-диагностикой
Недоступный remote	Контролируемый error без утечки credentials
Stop / force-stop	Task заканчивается stopped, post-delay marker отсутствует

6. Матрица клиентских конфигураций

Один набор тестов на разных способах хранения и выполнения

Профиль	База данных	Выполнение	Цель	Статус
core-sqlite-local	SQLite	Local server	Быстрый baseline	PASS
core-postgres-local	PostgreSQL 14.3	Local server	SQL dialect и чистые миграции	PASS
core-mysql-local	MySQL 8.4	Local server	MySQL dialect и migration path	PASS
core-mariadb-local	MariaDB 10.11	Local server	MySQL dialect на MariaDB	PASS
prod-postgres-runner	PostgreSQL 14.3	Persistent runner	Server -> DB -> remote runner	PASS
upgrade-sqlite-local	SQLite	v2.19.6 -> v2.19.7	In-place release upgrade	BUG
upgrade-postgres-local	PostgreSQL 14.3	v2.19.6 -> v2.19.7	In-place release upgrade	BUG

Воспроизводимость

Каждый manifest фиксирует release tag, source commit, edition, DB image, execution mode и capabilities. Фактические image digests, архитектура и выбранный профиль записываются в Allure environment.

Почему выбраны эти версии БД

MySQL 8.4 и MariaDB 10.11 совпадают с образами upstream CI Semaphore v2.19.7. PostgreSQL 14.3 соответствует используемому production-like профилю. Минимально поддерживаемые версии следует добавить отдельными compatibility jobs после фиксации support policy.

Единый интерфейс

```
test-environment/profile list
test-environment/profile up <profile>
test-environment/profile test <profile>
test-environment/profile down <profile>
test-environment/profile clean <profile> --yes
```

7. Обнаруженные дефекты и ограничения

Отдельно подтверждённые продуктовые баги, дефекты документации и конфигурационные контракты

BLOCKER: upgrade v2.19.6 -> v2.19.7

На SQLite и PostgreSQL новый release запускается и читает проектные данные, но GET /api/project/{id}/keys возвращает 400. Server log: gorp: no fields [task_id expire_at] in type AccessKey.

Подтверждённая причина

- v2.19.6 применяет миграцию v2.20.1 и добавляет access_key.task_id и access_key.expire_at;
- v2.19.7 удаляет migration entries, файлы миграций и соответствующие поля Go-модели;
- уже применённая схема не откатывается, поэтому gorp не может сопоставить существующие колонки;
- ошибка подтверждена одинаково на двух dialect и возникает внутри server, а не в тестовой DTO.

Влияние: безопасное обновление заблокировано для существующих установок; управление access keys не работает, а task flows с credentials нельзя считать пригодными после upgrade.

Дефекты API-документации

Расхождение	Практический эффект
GET schedules отсутствует	Router и живой API поддерживают GET/HEAD collection endpoint, OpenAPI описывает только POST
sort/order ошибочно required	Живые запросы без параметров возвращают 200; generated clients получают лишнее ограничение
Request required fields не заданы	Пустые тела выглядят валидными по схеме, хотя server требует конкретные поля
Router шире спецификации	Не полностью описаны task stages, refs, cron validation, permissions, secret storages и другие операции

Не классифицировано как баг

Global runner после авторегистрации имеет is_default=false и не получает задачи без tag, пока его не назначить default. Удаление пользователя после login history также остаётся наблюдаемым ограничением. Для обоих случаев нужно сначала подтвердить ожидаемый продуктовый контракт.

8. Качество реализации и готовность к передаче

Что уже сделано, чтобы следующий инженер не зависел от автора системы

Критерий	Реализовано
Читаемость	Domain API и steps разделены по группам; тесты используют chaining только там, где он улучшает сценарий
Тестовые данные	Typed fixtures отдельно от steps; deterministic seed и понятная команда replay
Preconditions	Login и стабильные fixture users подготавливаются до теста через typed state
Диагностика	Понятные exceptions, HTTP attachments, task output при ошибке, server logs для upgrade failure
Секреты	Централизованная redaction и прямые проверки отсутствия plaintext
Очистка	LIFO cleanup; primary failure не теряется; destructive clean требует явного --yes
Архитектура	64 self-tests контролируют ownership, isolation, dependencies и fixture boundaries
Документация	README, API map, component map, config strategy, smoke и upgrade reports

Материалы проекта

Артефакт	Расположение
План	semaphore-ui-testing-assessment-plan.md
Карта компонентов	semaphore-testing-component-map.md
Полный реестр issues	outputs/issues-assessment/semaphore-issues-register.xlsx
Карта API	test-environment/api-map.md
Матрица конфигураций	test-environment/configuration-testing-overview.md
Smoke-отчёт	test-environment/smoke-report.md
Upgrade-дефект	test-environment/upgrade-report.md

Готовность

Проект уже можно передавать для расширения API/UI coverage. До полноценного handoff остаются CI pipeline, короткий onboarding guide и согласование владельцев nightly/release результатов.

9. Рекомендуемые следующие шаги

Порядок основан на риске и стоимости поддержки

#	Направление	Результат	Приоритет
1	Оформить upgrade bug	Передать воспроизведение владельцу Semaphore; определить исправленный release; оставить job красным	Сейчас
2	CI baseline	SQLite core на PR; PostgreSQL + runner и SQL matrix nightly; artifacts всегда сохранять	P0
3	Schedules	Cron validation, timezone/DST, run once, persistence task params и exactly-once behavior	P1
4	SSH Git	SSH key, known_hosts, custom CA/config, submodules и отсутствие утечек	P1
5	External auth	OIDC + reverse proxy/subpath, затем LDAP TLS; негативный mapping и RBAC после входа	P1
6	Runner resilience	Disconnect/reconnect, capacity, queue reassignment и one-off runner	P2
7	Release coverage	Исправленный upgrade для MySQL/MariaDB, backup/restore, package/Helm smoke	P2
8	UI e2e	Небольшой набор: login, project/task run, schedule editing и критичная диагностика	P2

Предлагаемое распределение запусков

Частота	Набор	Цель
Каждый PR	Framework self-tests, formatting, core-sqlite-local	Быстрое обнаружение регрессий
Nightly	PostgreSQL local, MySQL, MariaDB, remote runner	Конфигурационная совместимость
Release	N-1 -> current, packages/Helm, backup/restore, amd64/arm64	Клиентский путь установки и обновления
Feature schedule	OIDC, LDAP, HA, dynamic runner	Тяжёлые специализированные профили

Критерий успеха следующего этапа

Новый инженер с чистой машины поднимает профиль, запускает suite, понимает причину типового падения и добавляет новый доменный тест без устных подсказок.

10. Приложение: быстрый запуск

Команды для локальной проверки и диагностики

Базовый профиль

```
test-environment/profile up core-sqlite-local
test-environment/profile test core-sqlite-local
test-environment/profile ps core-sqlite-local
test-environment/profile down core-sqlite-local
```

SQL-матрица

```
test-environment/profile up core-postgres-local
test-environment/profile test core-postgres-local

test-environment/profile up core-mysql-local
test-environment/profile test core-mysql-local

test-environment/profile up core-mariadb-local
test-environment/profile test core-mariadb-local
```

Remote runner

```
test-environment/profile up prod-postgres-runner
test-environment/profile test prod-postgres-runner
```

Upgrade reproduction

```
test-environment/profile upgrade-test upgrade-sqlite-local
test-environment/profile down upgrade-sqlite-local
test-environment/profile upgrade-test upgrade-postgres-local
```

Локальный quality gate

```
JAVA_HOME=/opt/homebrew/opt/openjdk@21 ./gradlew spotlessCheck frameworkTest
```

Важно

Все профили публикуют Semaphore на localhost:3000, поэтому запускаются последовательно. down сохраняет volumes; clean удаляет только volumes выбранного профиля и требует --yes.

Итог

Базовая система тестирования создана и уже принесла практический результат: подтвердила критические пользовательские контракты на нескольких конфигурациях и обнаружила блокирующий дефект реального release upgrade. Следующий этап - встроить эту основу в CI и последовательно расширять покрытие по карте рисков.